Student Task Management System

A PROJECT REPORT

Submitted by

Shalini Pandurang Shanbhag - 1BM24MC086

in partial fulfillment for the award of the degree of Master of Computer Applications

Under the guidance of

Internal Guide
Dr. D. N. Sujatha
Professor,
Dept. of Computer Applications
B. M. S. College of Engineering,
Bangalore-560019

External Guide
Mr. Ramkumar
Manager
Nyera Edutech & Innovations
Bangalore-560001

B. M. S. COLLEGE OF ENGINEERING
BANGALORE-19
JUNE -2026

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU, Approved by AICTE, Accredited by NAAC)

MASTER OF COMPUTER APPLICATIONS

This is to certify that Shalini Pandurang Shanbhag[1BM24MC086] has satisfactorily completed the requirements for Major Project - MMCPRJ401 entitled "Student Task Management System" as a part of the 4th Semester MCA course in this college during the year 2025.

Internal Guide

…….

Dr. D. N. Sujatha
Professor
B. M. S. College of Engineering
Bangalore- 560019

External Guide

…….

Mr. Ramkumar
Manager
Nyera Edutech & Innovations Pvt Ltd
Bangalore – 560001

Head of the Department

……..

Dr. S Uma
Professor and Head, Dept. of CA
Dept. of Computer Applications
B.M.S. College of Engineering
Bangalore- 560019

Principal

…….

Dr. Bheemsha Arya
Principal,
B.M.S. College of Engineering
Bangalore-560019

……..

Examiner 1

……

Examiner 2

DECLARATION

I hereby declare that this project entitled **"Student Task Management System"** carried out at **"Nyera Edutech & Innovations Pvt Ltd"** is a record of project work submitted to the **Department of Computer Applications, B.M.S College of Engineering**, affiliated to **Visvesvaraya Technological University** is an original work completed by me.

To the best of my knowledge, this report has not been submitted to any other college or university or published at any time prior to this.

Date: **Name: Shalini Pandurang Shanbhag**

Place: Bangalore **USN: 1BM24MC086**

ACKNOWLEDGEMENT

No achievement is possible without the help, support, and guidance of inspirational people and this project is incomplete without an acknowledgment of the gratitude of these individuals. Each one of them has contributed in their own unique and invaluable way to the undertaking of this project.

I take this opportunity to thank all the people who have been supportive throughout this report.

First, I would like to thank my external guide **Mr. Ramkumar**, Manager of Nyera Edutech & Innovations Pvt Ltd. Who has been a constant support in the process of completing my Internship.

Second, I would like to thank my internal guide **Dr. D. N. Sujatha**, Professor, Department of Computer Applications, for his constant support and guidance in preparing this report.

Next, I would like to thank my Head of the Department **Dr. S Uma** for her guidance and support.

I also wish to place my deep debt of gratitude to **Dr. Bheemsha Arya**, Principal, B. M. S. College of Engineering for his constant help.

I would also like to thank my parents and friends for their support and guidance.

Shalini Pandurang Shanbhag

USN: 1BM24MC

ABSTRACT

The project titled **"Student Task Management System"** is a modern web-based productivity and academic management system developed to help students efficiently organize tasks, monitor academic activities, and analyze their overall performance through an interactive dashboard. In today's academic environment, students often face difficulties in managing assignments, project deadlines, examinations, and daily study schedules due to the absence of a centralized and intelligent tracking platform. Traditional methods such as notebooks, spreadsheets, or simple reminder applications lack analytical capabilities, real-time accessibility, and performance monitoring features. This system aims to overcome these limitations by providing a secure, cloud-based solution that combines task management with productivity analytics.

The proposed system allows users to register and securely log in using authentication mechanisms. Students can create, update, categorize, prioritize, and track academic tasks based on deadlines and completion status. The application also provides productivity insights through visual dashboards, enabling users to analyze completed tasks, pending activities, overdue work, and category-wise performance trends. These analytical features help students improve time management, academic planning, and overall productivity.

The system is developed using modern full-stack technologies including Java Spring Boot for backend development, React.js for frontend development, and MySQL for database management. RESTful APIs are used for communication between frontend and backend components, while JWT (JSON Web Token) authentication ensures secure access control and protected user sessions. The application is deployed on cloud platforms, making it accessible anytime and from any location.

The project follows a modular and scalable software architecture that supports future enhancements such as AI-based task recommendations, mobile application integration, calendar synchronization, reminder notifications, and collaborative task management. The implementation of dashboards, analytics, authentication, and responsive design makes the system more advanced than traditional task management applications.

The "Student Task Management System" not only simplifies academic task management but also provides meaningful productivity insights that assist students in improving efficiency, reducing missed deadlines, and achieving better academic performance.

.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

1. INTRODUCTION

1.1 PROJECT DESCRIPTION

This project was undertaken at Nyera Edutech & Innovations Private Limited, a pioneering IT-based company leading the charge in technological innovation. With an unwavering commitment to providing comprehensive solutions that span a diverse range of domains, the organisation has set out to redefine the very fabric of the digital landscape. Its expertise encompasses a wide spectrum of services, each meticulously designed to empower businesses and individuals alike.

The project involved the design and implementation of Tasklytic — an AI-powered academic task management system developed as a final-year project. The primary objective was to create a scalable, responsive, and secure web application where students can register, authenticate, manage their academic tasks, track progress, and receive AI-generated study plans. The project was implemented using a modern JavaScript-based full-stack architecture comprising Node.js, Express.js, React.js (Vite), and MySQL — a robust combination for building performant, maintainable web applications.

The platform incorporates essential academic productivity features including user authentication and profile management, task and subtask creation with CRUD operations, AI-powered task breakdown using Google Gemini 2.5 Flash, a daily planner view, an Eisenhower priority matrix, productivity analytics with a heatmap grid, and automated email deadline reminders. An AI controller was also built to interface with the Gemini API, generating 4 to 6 actionable, time-estimated subtasks per assignment along with a personalised success strategy — providing students with structured guidance beyond a simple to-do list. Front-end development was executed using React.js for its component-based architecture and dynamic rendering capabilities, while a clean and fully responsive UI was implemented to enhance usability across devices.

The backend was powered by Node.js and Express.js, enabling a fast and non-blocking server environment. Data is stored and managed using MySQL, with the mysql2 driver handling connection pooling and structured relational queries. Routes are organised by domain — user, task, analytics, and AI — each protected by JWT-based authentication middleware to ensure secure session handling. APIs were structured according to REST principles, ensuring modularity and maintainability. A scheduled background service built with node-cron runs each morning to identify tasks due the following day and dispatches richly formatted HTML reminder emails via Nodemailer, automating the notification workflow without any manual intervention.

This project has been a valuable hands-on learning experience. It not only strengthened the understanding of full-stack web development but also exposed the development team to real-world software design principles such as modular coding, secure authentication, API integration with large language models, database modelling, and UI/UX design. Working on this project improved problem-solving skills and time management — key competencies required in professional development environments.

Overall, Tasklytic represents the successful application of theoretical knowledge to a practical, real-world academic problem. It demonstrates how AI technology can be leveraged to deliver targeted, intelligent, and user-friendly solutions that meet specific domain needs. The project has laid a solid foundation for future enhancements, including the integration of a conversational chatbot assistant, mobile application development, and collaborative study group features — all aimed at improving the overall user experience and academic value of the platform

1.2 MOTIVATION

The rapid evolution of academic technology has fundamentally reshaped how students organise, plan, and execute their coursework, with a notable shift toward intelligent digital tools. Academic task management, as a critical component of student productivity, demands a platform that is both user-centric and technologically advanced. This led to the conceptualisation and development of Tasklytic — an AI-powered academic task management system capable of meeting these emerging needs. The technical motivation behind this project lies in building a responsive, secure, scalable, and modular web application using a modern JavaScript-based full stack — a widely adopted and powerful combination of technologies that support end-to-end development. The stack is composed of four core technologies:

- **React.js (Vite)** – A powerful frontend library for building dynamic, component-driven user interfaces.
- **Node.js** – A JavaScript runtime environment that powers scalable server-side applications.
- **Express.js** – A minimalist and fast Node.js web application framework for building REST APIs.
- **MySQL** – A reliable relational database for structured storage of users, tasks, subtasks, and analytics data.

This technology stack enables full-stack development using a single language — JavaScript — which allows better team collaboration and easier debugging. It supports single-page applications (SPAs), ensuring fluid navigation and minimal full-page reloads, critical for maintaining focus and engagement in a productivity tool.

Addressing Real-World Academic Productivity Challenges:

1. Scalability and Structured Data Management

MySQL with the mysql2 driver allows Tasklytic to handle growing volumes of structured data including user accounts, tasks, subtasks, analytics records, and heatmap activity. Its relational schema ensures data integrity across linked entities, making it well-suited for tracking hierarchical relationships such as tasks containing multiple subtasks, each with their own completion state and time estimates.

2. High Performance and User Experience

React's virtual DOM offers efficient UI updates without re-rendering the entire page, ensuring a fast and smooth experience as students interact with the task list, daily planner, Eisenhower matrix, and analytics dashboard. React's component-based architecture also simplifies code reusability and state management, allowing pages like TaskDetail, Dashboard, and Analysis to share common logic cleanly.

3. Security and User Authentication

With Express.js and Node.js powering the backend, the application benefits from robust routing and middleware integration. Secure user authentication and session management were implemented using JWT (JSON Web Tokens), enabling protected routes, encrypted communication between the frontend and backend, and safe handling of sensitive user data such as credentials and security questions. A forgot-password and reset-password flow was also implemented via Nodemailer for complete account security.

4. AI Integration and Intelligent Extensibility

A key motivation for Tasklytic was the integration of generative AI into the academic workflow. The Google Gemini 2.5 Flash API was embedded directly into the task lifecycle, allowing the system to automatically decompose any assignment into 4–6 actionable, time-estimated subtasks along with a personalised success strategy and effort-level classification. This demonstrates how the modular Node.js ecosystem supports seamless integration with powerful third-party AI services, transforming a basic task manager into an intelligent study companion.

5. Automation and Proactive Notifications

The platform leverages node-cron to run scheduled background jobs that identify tasks due the following day and automatically dispatch richly formatted HTML reminder emails to users via Nodemailer. This real-time automation reduces missed deadlines without any manual intervention, demonstrating how backend scheduling can enhance user experience beyond the visible interface.

6. Developer Productivity and Maintainability

By using a unified JavaScript stack across both frontend and backend, the project benefits from reduced context switching and faster development cycles. The backend routes are organised by domain — user, task, analytics, and AI — each with dedicated controllers and services, following a clean modular folder structure. This separation of concerns improves long-term maintainability and makes the codebase straightforward to extend with future features

1.3 EXISTING AND PROPOSED SYSTEM

The existing academic task management solutions available to students are typically built on generic productivity platforms such as Microsoft To-Do, Notion, Google Tasks, or Trello. These platforms support basic functionalities such as task creation, due date setting, and simple progress tracking. However, they lack specialised features tailored for academic users, such as AI-driven task breakdown, course-category organisation, study effort estimation, or deadline-aware email notifications. Most existing systems offer a standard template with limited scope for academic personalisation, which affects student engagement and study effectiveness.

Additionally, productivity tracking features such as heatmap grids, Eisenhower priority matrices, and per-subject analytics are either absent or locked behind premium subscription tiers. Integration with AI for intelligent study guidance is either non-existent or requires expensive third-party add-ons. These platforms provide a good general foundation but do not offer the flexibility or intelligence needed to support a student's academic workflow end to end.

Disadvantages of the Existing System:

- Generic platforms are not designed for academic use and do not support course-based task categorisation, subtask time estimation, or study strategy generation.
- Most tools lack AI integration, leaving students responsible for manually breaking down complex assignments without guidance.
- Productivity analytics such as completion heatmaps, effort-level tracking, and workload distribution charts are rarely available in free-tier tools.
- Automated deadline reminder emails are either missing or require manual configuration through third-party integrations.
- Priority frameworks like the Eisenhower Matrix are not built into existing tools and must be manually replicated using workarounds.

- Existing platforms offer no daily planner view scoped to academic subtasks, making it difficult to focus on immediate workload.
- Data is stored across disconnected apps, creating fragmentation in a student's productivity system and reducing overall consistency.

1.3.2 PROPOSED SYSTEM

The proposed system is Tasklytic — a dedicated AI-powered academic task management platform tailored specifically for students managing coursework, assignments, and study schedules. Unlike generic productivity tools such as Notion or Trello, this custom-built platform aims to provide a targeted, intelligent, and student-focused experience that emphasises structured planning, AI assistance, and automated support. The project has been developed using a modern JavaScript full stack comprising React.js (Vite), Node.js, Express.js, and MySQL, ensuring the system is built on a scalable, maintainable, and efficient technology base. The goal is to create not just a to-do list, but a complete digital academic companion that guides students from assignment receipt through to completion — offering AI-generated study plans, visual progress analytics, and proactive deadline management.

Objectives of the Proposed System:

- To provide a centralised platform for students to create, organise, and track all academic tasks and subtasks in one place.
- To integrate Google Gemini AI to automatically break down assignments into structured, time-estimated subtasks with personalised study strategies.
- To improve student engagement and consistency through features like a productivity heatmap, daily planner, Eisenhower matrix, and automated email reminders.

Advantages of the Proposed System:

- **Academic-Focused Experience:** Tailored layout, course-based categorisation, and AI-driven recommendations specific to student workflows.
- **Intelligent Task Breakdown:** Gemini AI analyses each task and generates 4–6 actionable subtasks with time estimates and a success strategy, reducing cognitive load on students.
- **Automated Notifications:** A cron-based email reminder system dispatches deadline alerts the day before tasks are due, without requiring any manual action from the user.
- **Scalable Design:** The modular architecture can be expanded in future to include collaborative study groups, mobile applications, or integration with university learning management systems.

System Features:

- **User Registration/Login:** Secure sign-up with JWT-based authentication, password encryption via bcrypt, and a forgot/reset password flow using Nodemailer.
- **Task & Subtask Management:** Full CRUD operations for academic tasks with support for title, description, category, due date, priority, and status tracking.
- **AI-Powered Task Breakdown:** Google Gemini 2.5 Flash generates structured subtasks, time estimates, effort level, and a personalised study tip for any assignment.
- **Daily Planner:** A focused view surfacing subtasks due on the current day, helping students prioritise their immediate workload.
- **Eisenhower Matrix:** Visual quadrant-based task prioritisation by urgency and importance, enabling deliberate decision-making.
- **Productivity Analytics & Heatmap:** Dashboard charts showing completion trends, category distributions, and a GitHub-style daily activity heatmap for longitudinal progress tracking.

- **Automated Email Reminders:** Scheduled background jobs (node-cron) identify tasks due the next day and send formatted HTML reminder emails automatically.
- **Settings & Profile Management:** Users can update personal details, change passwords, and manage security questions from a dedicated settings page.
- **Responsive Design:** Fully optimised for desktops, tablets, and mobile devices

1.4 HARDWARE AND SOFTWARE REQUIREMENTS

1.4.1 HARDWARE SPECIFICATION

Table 1 Hardware Specification

Component	Specification
Processor (CPU)	Intel Core i5 / i7 or AMD Ryzen 5 / 7 (Quad-Core or higher)
RAM	Minimum 8 GB (Recommended: 16 GB for smooth multitasking)
Storage	256 GB SSD or higher (Recommended: 512 GB SSD for fast read/write operations)
Operating System	Windows 10 / 11 (64-bit), Ubuntu 20.04+, or macOS 10.15+
Graphics	Integrated GPU (sufficient for web development); optional discrete GPU for design work
Network	Stable broadband internet connection (≥ 20 Mbps) for package installation, API testing, and deployment

1.4.2 SOFTWARE SPECIFICATION

Table 2 Software Specification

Category	Software / Technology (with Version)
Frontend	React.js v18.x (Vite 5.x build tool)
	JavaScript ES2022
	Axios v1.x
	HTML5, CSS3
	React Router DOM v6.x
Backend	Node.js v18.17.1
	Express.js v5.2.1
Database	MySQL v8.0+ (with mysql2 v3.x driver)
ORM / Query	mysql2 with connection pooling (no ORM)
Authentication	JSON Web Token (JWT) v9.0.x + bcrypt v6.x
AI Integration	Google Gemini 2.5 Flash API (@google/generative-ai v0.24.1)
Email Service	Nodemailer v8.x
Scheduler	node-cron v4.x
Code Editor	Visual Studio Code v1.90.0
Version Control	Git v2.43.0 + GitHub
Web Server	Node.js Server with Express Middleware
Operating System	Windows 10 / 11, Ubuntu 20.04 LTS, or macOS
Browser Compatibility	Google Chrome v138.0.7204, Mozilla Firefox v140.0, Microsoft Edge v138.0.3351.65, Safari v18.5

2. SYSTEM ANALYSIS

2.1 FEASIBILITY STUDY

A feasibility study is an essential step in evaluating whether a project can be successfully designed, developed, and deployed. The feasibility for Tasklytic — an AI-powered academic task management system — has been assessed in the following dimensions:

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility
- Scheduling Feasibility

2.1.1 TECHNICAL FEASIBILITY

The proposed Tasklytic platform is technically feasible, as it utilises a mature and reliable modern JavaScript full stack comprising React.js (Vite), Node.js, Express.js, and MySQL. Each of these technologies is open-source, widely adopted, and well-supported, making them ideal for building scalable and maintainable web applications.

- **MySQL** with the mysql2 driver provides structured, relational data storage for users, tasks, subtasks, analytics records, and heatmap activity. Its relational schema ensures data integrity across linked entities and supports complex queries for analytics and scheduling operations.
- **Node.js and Express.js** form the server-side foundation, offering a lightweight, high-performance runtime environment and a robust framework for handling API routes, authentication middleware, scheduled jobs, and server logic.
- **React.js (Vite)** enables the creation of a dynamic and responsive user interface, providing a smooth user experience with real-time updates, component-based architecture, and minimal page reloads.
- The stack allows full-stack development using a single language — JavaScript — simplifying the development process and enhancing long-term maintainability.
- The system leverages third-party open-source libraries for features such as JWT-based authentication, password hashing via bcrypt, automated email delivery via Nodemailer, background scheduling via node-cron, and AI-powered task breakdown via the Google Gemini 2.5 Flash API.
- Deployment is technically achievable using modern cloud platforms such as Vercel and Render, which support continuous deployment, auto-scaling, and version rollbacks, making the system production-ready with minimal infrastructure overhead.

Given these technical factors, the system is fully capable of meeting performance expectations and supporting future scalability.

2.1.2 OPERATIONAL FEASIBILITY

Operationally, the system is highly feasible, offering a practical and user-friendly solution for students managing academic workloads.

- Students can register, log in securely, create and organise tasks by course category, generate AI-powered subtask breakdowns, track daily progress through the planner, and receive automated email reminders before deadlines — all from a single platform.

- The AI integration via Google Gemini reduces the cognitive effort required to plan assignments by automatically producing structured study plans, making the platform accessible even to students unfamiliar with productivity frameworks.
- Secure JWT-based authentication ensures that only authorised users can access their data, with bcrypt-encrypted passwords and a self-service password reset flow via email for complete account security.
- The Eisenhower Matrix and productivity heatmap give students visual tools to prioritise tasks and reflect on study consistency, reducing the reliance on external tracking spreadsheets or paper planners.
- The automated cron-based email reminder system operates entirely in the background, dispatching deadline alerts without any manual intervention, reducing missed submissions and improving academic outcomes.
- The settings module allows users to independently manage their profile, password, and security preferences, minimising administrative overhead.

2.1.3 ECONOMIC FEASIBILITY

The system is economically viable and cost-effective, particularly suitable for individual student developers and academic institutions.

- The entire solution is developed using open-source technologies, eliminating the need for costly proprietary software or licensing fees.
- MySQL can be hosted on free-tier cloud database services, and platforms such as Vercel allow initial frontend and backend deployment without financial burden, with the option to scale later based on usage growth.
- The Google Gemini API provides a free usage tier sufficient for development and moderate production use, making AI integration accessible without significant cost.
- The use of a modular codebase with clearly separated controllers, services, and routes reduces long-term development and maintenance costs by making individual components easy to update or replace independently.
- Nodemailer with a standard email provider eliminates the need for paid transactional email services during early deployment stages.
- Automated task reminders, analytics, and AI guidance reduce the need for external tutoring tools or premium productivity subscriptions, delivering tangible value to students at no recurring cost.

2.1.4 SCHEDULING FEASIBILITY

The project was planned and executed using the Agile development methodology, a flexible and iterative approach that promotes rapid delivery, continuous feedback, and adaptive planning. Agile allowed the development team to efficiently respond to changing requirements while ensuring that each stage of development was systematically addressed. The overall development cycle was structured into clearly defined and sequential phases — beginning with requirement gathering, where user stories, technical specifications, and feature priorities were documented, followed by UI/UX design, where page wireframes were developed to ensure an intuitive interface aligned with student workflows.

Backend development focused on building a secure, modular architecture supporting user authentication, task and subtask management, AI integration, analytics, and scheduled email services. In parallel, frontend integration ensured that all pages were responsive and seamlessly connected to the backend through RESTful APIs. Each module was developed in sprint cycles, with individual sprints dedicated to specific components such as user registration and login, task CRUD operations, AI task breakdown, daily planner, Eisenhower matrix, heatmap analytics, and the cron-based email reminder service.

GitHub was used for version control, ensuring code consistency through branching strategies, pull requests, and frequent commits. Regular code reviews were conducted to maintain quality and identify issues early. Unit testing and integration testing were carried out across sprints to ensure all modules functioned correctly. Documentation was maintained alongside development, including API documentation and deployment guides, to support future maintenance.

Effective time management, modular task distribution, and adherence to Agile principles allowed the team to deliver a high-quality, production-ready system within the proposed timeline. The successful implementation of Tasklytic demonstrates the efficiency of Agile methodology in managing a multi-featured, AI-integrated web application while maintaining flexibility, scalability, and quality throughout the development lifecycle.

3. PROJECT PLAN

Table 3 Project Plan

Phase	Description
1. Requirement Analysis	Define project goals, functional/non-functional requirements, and feature list.
2. UI/UX Design	Design wireframes, page layouts, and user interaction flow.
3. Environment Setup	Set up React (Vite), Node.js, Express.js, MySQL environment, GitHub repo, and local database.
4. Backend Development	Create Express.js REST APIs, MySQL schemas, authentication logic, and cron job services.
5. Frontend Development	Build responsive React.js UI and integrate with backend APIs using Axios.
6. AI Integration Module	Implement Google Gemini API controller for AI-powered task breakdown and subtask generation.
7. Testing & Debugging	Perform unit testing, API testing, integration testing, and bug fixes.
8. Deployment & Hosting	Deploy on Vercel (frontend + backend) with cloud-hosted MySQL database.
9. Documentation & Report	Finalise project report, testing documents, and user manual.
10. Final Presentation Prep	Create PPT, demo setup, and viva readiness.

3.1 WORKFLOW

The development of Tasklytic was carried out in a structured and phased manner to ensure systematic progress, efficient task management, and high-quality output. Below is the detailed explanation of each phase involved in the workflow:

Phase 1: Requirement Analysis (1 week)

This initial phase involved understanding the project's objective, scope, and expectations. The team gathered both functional requirements (e.g., user login, task management, AI breakdown, email reminders) and non-functional requirements (e.g., performance, security, responsiveness). A clear feature list was created and user roles were identified. This phase laid the foundation for the rest of the development cycle.

Phase 2: UI/UX Design (2 weeks)

Once the requirements were finalised, the focus shifted to designing the user interface and user experience. Wireframes and mockups were created for all major pages including the login/register screens, dashboard, task list, task detail, daily planner, Eisenhower matrix, analytics, and settings. The design process prioritised usability, clarity, and a clean academic-themed visual aesthetic.

Phase 3: Environment Setup (1 week)

With the design and requirements in place, the technical environment was prepared. React.js (Vite), Node.js, Express.js, and MySQL were configured. A repository was created on GitHub for version control and collaboration. The local MySQL database was set up and development environments were configured for parallel development.

Phase 4: Backend Development (3 weeks)

This phase focused on implementing the server-side logic using Node.js and Express.js. Key REST APIs were built for user registration, login, password reset, task CRUD operations, subtask management, analytics, and AI integration. MySQL table schemas were defined for users, tasks, subtasks, and analytics records. JWT was used for secure authentication and route protection, and bcrypt was used for password hashing.

Phase 5: Frontend Development (3 weeks)

The React.js-based frontend was developed to ensure a smooth, interactive user experience. Components and pages were built for login/register, dashboard, task list, task detail, add/edit task, daily planner, Eisenhower matrix, analysis, and settings. React Router handled navigation between pages and Axios was used to consume backend APIs.

Phase 6: AI Integration Module (2 weeks)

A dedicated AI controller was developed to interface with the Google Gemini 2.5 Flash API. This module accepts a task's title, description, and course category, submits a structured prompt to the Gemini model, and parses the returned JSON to extract 4–6 actionable subtasks with time estimates, a personalised success strategy, and an effort-level classification. A deterministic fallback was implemented to ensure the feature remains functional if the API is unavailable.

Phase 7: Testing & Debugging (2 weeks)

After the core functionalities were built, the application underwent multiple levels of testing:

- Unit testing for individual functions and components
- API testing using Postman
- Integration testing between frontend and backend
- UI/UX testing across different devices and browsers

Bugs were identified and resolved at this stage to ensure system reliability and usability.

Phase 8: Deployment & Hosting (1 week)

Upon successful testing, the application was deployed. The backend and frontend were hosted on Vercel and a cloud-hosted MySQL instance was used for database management. Environment variables, JWT secrets, Gemini API keys, and email credentials were configured appropriately to ensure a smooth production setup.

3.2 GANTT CHART

A Gantt chart plays a crucial role in project scheduling by offering a clear visual representation of all development activities over a defined timeline. It allows project managers and developers to effectively map out tasks, showing each phase's start and end times with precision. This helps in avoiding scheduling conflicts, ensuring that each development activity is logically placed within the available time frame.

By plotting phases in chronological order, the Gantt chart supports real-time visibility into ongoing and upcoming tasks, making it easier to plan and prioritise effectively. It enables simultaneous tracking of

multiple workstreams — such as frontend development running in parallel with backend API development — which is especially valuable in full-stack projects with interdependent modules.

Furthermore, Gantt charts enhance resource management by clearly displaying the allocation of development effort across phases. They make it easy to identify task dependencies, ensuring that activities such as AI integration and frontend wiring are scheduled only after the foundational backend APIs are in place. Their visual nature simplifies spotting bottlenecks or idle periods, allowing the team to make proactive adjustments and maintain momentum throughout the development cycle.

Phase / Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Requirement Analysis	✓	✓													
System Design			✓	✓											
Database Design				✓	✓										
Backend – User Module					✓	✓									
Backend – Task Module						✓	✓	✓							
Backend – Analytics & AI								✓	✓						
Frontend – Auth Pages							✓	✓							
Frontend – Dashboard								✓	✓	✓					
Frontend – Tasks & Planner										✓	✓				
Frontend – Analysis & AI											✓	✓			
Integration Testing												✓	✓		
System & UAT Testing													✓		
Cloud														✓	

Deployment															
Documentation & Report														✓	✓

Figure 1 Gantt Chart

Project Timeline:

- **Week 1 — Project Initiation:** Define the overall project scope, identify objectives, assign roles, and set up planning tools. Finalise the technology stack (React, Node.js, Express.js, MySQL, Gemini AI) and conduct the kickoff meeting.
- **Weeks 2–3 — Requirement Analysis:** Gather functional and non-functional requirements. Prepare use-case diagrams, flowcharts, and requirement specification documents. Finalise the prioritised feature list and define user roles.
- **Weeks 4–5 — UI/UX Design:** Design low- and high-fidelity wireframes. Establish design guidelines including colour themes, typography, and layout consistency. Create responsive mockups for key pages including dashboard, task detail, planner, matrix, and analytics.
- **Weeks 6–7 — Frontend Development:** Begin implementing the UI in React.js with Vite. Develop reusable components (layout, navigation, cards, modals) and implement page-level routes. Ensure responsive and accessible design across devices.
- **Weeks 8–9 — Backend Development:** Set up the Node.js and Express.js server. Create RESTful APIs for user management, task and subtask operations, analytics, and AI integration. Implement JWT authentication middleware, bcrypt password hashing, and email services via Nodemailer.
- **Week 10 — Database Design:** Model the MySQL database with tables for users, tasks, subtasks, and analytics. Define relationships, constraints, and indexes. Integrate the database with the backend and seed initial test data.
- **Weeks 11–12 — Integration & AI Module:** Connect the frontend to backend APIs using Axios. Implement the Google Gemini AI controller for task breakdown. Test complete user flows including registration, task creation, AI generation, planner, and analytics.
- **Weeks 13–14 — Testing and QA:** Perform unit testing on components and APIs. Conduct integration and system testing to validate full user flows. Identify and fix bugs related to UI behaviour, data handling, cron scheduling, and AI parsing logic.
- **Week 15 — Deployment and Hosting:** Deploy the frontend and backend to Vercel. Configure cloud MySQL hosting, environment variables, and production builds. Conduct final round testing on the live environment and prepare the system for presentation.

4. SOFTWARE REQUIREMENTS SPECIFICATIONS (SRS)

4.1 FUNCTIONAL REQUIREMENTS

1. User Management

- **User registration and login:** Enables new users to create accounts using their email and password, allowing personalised experiences and secure access to the platform.
- **Profile management:** Allows users to update personal details such as name, email, and profile information from the settings page.
- **Password recovery via email:** A secure mechanism to reset forgotten passwords through an email-based OTP or reset link flow, enhancing account accessibility.
- **Security question management:** Users can set and update a personal security question as an additional layer of account verification and recovery.
- **Change password:** Authenticated users can update their password directly from the settings page with current password verification.

2. Task Management

- **Create and manage tasks:** Users can add academic tasks with details including title, description, course category, due date, priority level, and completion status.
- **View and filter tasks:** Tasks can be browsed, searched, and filtered by category, priority, status, or due date for quick access to relevant assignments.
- **Edit and delete tasks:** Users can update any task attribute or remove tasks that are no longer relevant.
- **Task detail view:** Displays comprehensive task information including all associated subtasks, AI-generated tips, effort level, and progress indicators.

3. Subtask Management

- **Add and manage subtasks:** Each task can be broken down into individual subtasks with titles and estimated time, enabling granular progress tracking.
- **Mark subtasks complete:** Users can check off individual subtasks, with overall task progress updating accordingly.
- **Edit and delete subtasks:** Subtasks can be modified or removed independently without affecting the parent task.
- **Daily subtask view:** Surfaces subtasks due or scheduled for the current day in a focused daily planner interface.

4. AI-Powered Task Breakdown

- **Automated subtask generation:** The Google Gemini 2.5 Flash API analyses a task's title, description, and course category to generate 4–6 structured, time-estimated subtasks automatically.
- **Personalised success strategy:** Alongside subtasks, the AI provides a tailored study tip and an overall effort-level classification (low, medium, or high).
- **Fallback mechanism:** A deterministic fallback ensures the feature remains functional and returns sensible default subtasks if the Gemini API is unavailable.

5. Daily Planner

- **Day-scoped task view:** Displays all subtasks due or assigned to the current day, helping students focus on immediate priorities without distraction from distant deadlines.
- **Completion tracking:** Users can mark daily subtasks as complete directly from the planner view, maintaining progress records.

6. Eisenhower Matrix

- **Priority quadrant visualisation:** Tasks are displayed across four quadrants based on urgency and importance, giving students a clear framework for deciding what to act on first.
- **Interactive task management:** Users can interact with tasks directly within the matrix view to update priority or status.

7. Productivity Analytics & Heatmap

- **Completion trend charts:** The analytics dashboard visualises task completion rates, workload distribution by category, and progress over time.
- **Activity heatmap:** A GitHub-style grid displays daily task activity density over weeks and months, providing a longitudinal view of study consistency.
- **Effort and category breakdown:** Visual breakdowns show how effort is distributed across different courses or subjects, helping students identify imbalances.

8. Automated Email Reminders

- **Scheduled deadline alerts:** A cron-based background service runs daily, identifies tasks due the following day, and automatically dispatches formatted HTML reminder emails to the relevant users.
- **Rich email formatting:** Reminder emails include task titles, categories, due dates, and a call-to-action, providing clear and actionable information.

9. Settings & Account Management

- **Profile update:** Users can edit their display name and email from the settings page.
- **Password change:** Secure in-app password update with current password verification.
- **Security question management:** Users can view and update their security question and answer at any time.
- **Notification preferences:** Users can manage email notification settings from the settings panel.

4.2 NON-FUNCTIONAL REQUIREMENTS

1. Performance

- The system must efficiently handle multiple concurrent user sessions without lag, ensuring smooth interaction across task management, AI generation, and analytics pages.
- Fast loading times for all pages, with optimised API calls, connection pooling via mysql2, and efficient database queries for task retrieval and analytics aggregation.
- The Gemini AI integration must handle response parsing gracefully, with timeouts and fallback logic ensuring the interface remains responsive regardless of external API latency.

2. Scalability

- The application is designed using a modular, domain-separated architecture — with independent controllers, services, and routes for user, task, analytics, and AI modules — allowing future expansion without reengineering the entire system.
- The Express.js backend can be horizontally scaled by deploying additional server instances as user traffic increases.
- The MySQL database should support indexing and query optimisation strategies to maintain performance as the volume of tasks, subtasks, and analytics records grows.
- Frontend and backend services are independently deployable, enabling feature updates to either layer without disrupting the other.

3. Security

- Use of HTTPS for all user interactions ensures that data transmitted between client and server is encrypted, protecting credentials and personal information from interception.
- JWT-based authentication secures all protected API routes, with tokens verified via middleware before any user-specific data is accessed or modified.
- Passwords are hashed using bcrypt before storage, ensuring that plaintext credentials are never held in the database.
- Email-based password reset tokens are time-limited and single-use, preventing replay attacks on the account recovery flow.
- Environment variables store all sensitive credentials including JWT secrets, Gemini API keys, database credentials, and email service passwords, keeping them out of the codebase.

4. Usability

- Clean and consistent UI/UX design ensuring ease of navigation for students of all technical backgrounds.
- Fully responsive design delivering an optimal experience across desktops, tablets, and mobile devices.
- The AI task breakdown feature reduces cognitive load by generating structured study plans automatically, making the platform useful even without prior productivity training.
- Visual tools such as the Eisenhower matrix, heatmap, and daily planner provide intuitive, at-a-glance understanding of workload and progress.

5. Maintainability

- Use of version control via Git and GitHub with branching strategies for feature development and bug fixes.
- The codebase follows a modular folder structure with clearly separated concerns — routes, controllers, services, middleware, and models — aligned with MVC principles.
- API endpoints are organised and documented, supporting future integration with mobile applications or third-party tools.
- Error handling middleware in Express.js ensures consistent error responses across all API routes, simplifying debugging and monitoring.

6. Compatibility

- The web application must function correctly across all major browsers including Chrome, Firefox, Safari, and Edge.
- Compatible with different operating systems including Windows, macOS, Android, and iOS.

- The REST API architecture ensures that backend services can support future mobile application integrations without requiring significant rework.
- The responsive UI automatically adjusts to different screen resolutions, ensuring consistent usability across smartphones, tablets, and large monitors.

5. SYSTEM DESIGN

5.1 CLASS DIAGRAM

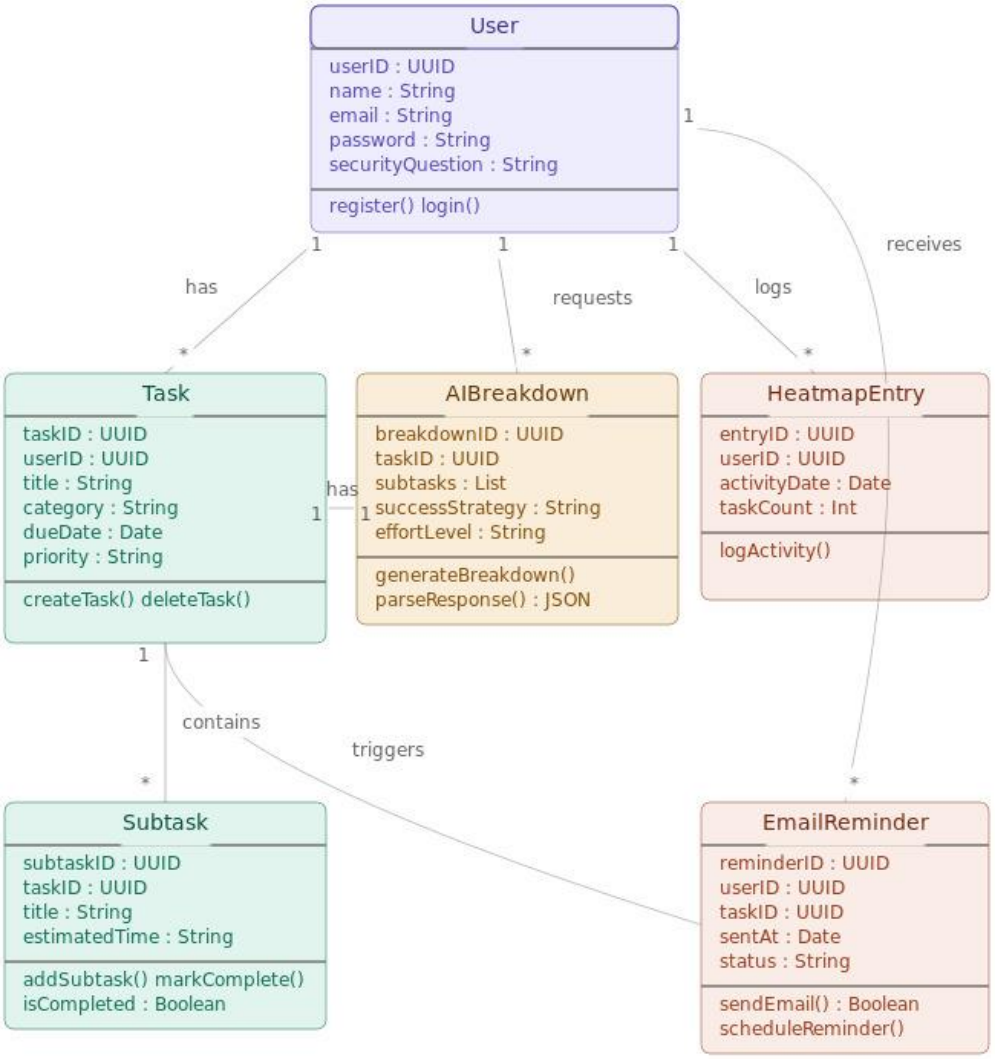


Figure 2: Class Diagram of System Architecture

The use case diagram in Figure 2 illustrates the structure and interrelationships between key backend components of the system, including core classes such as User, Product, Order, Cart, Payment, Wishlist, and ShippingDetail. Each class encapsulates relevant attributes (e.g., name, price, status) and methods (e.g., addToCart(), makePayment(), trackOrder()), enabling clear modeling of platform behavior and functionality. Relationships among entities are well-defined—for example, a User can place multiple Orders, and each Order can consist of multiple OrderItems—demonstrating one-to-many associations that reflect real-world interactions. This diagram serves as a blueprint for backend development, helping developers understand how different entities collaborate, manage data flow, and maintain consistency across the system. It provides a high-level yet precise view of the business logic, facilitating effective implementation, scalability, and future enhancements.

In addition to outlining entity relationships, the diagram also highlights the separation of concerns within the system, ensuring modularity and ease of maintenance. For instance, the inclusion of classes like Wishlist and ShippingDetail allows for extended user engagement and order tracking without overloading the core User or Order classes. Similarly, the Cart and Payment classes encapsulate their own logic for managing product selection and transaction handling, promoting reusable and testable components.

5.2 USE CASE DIAGRAM

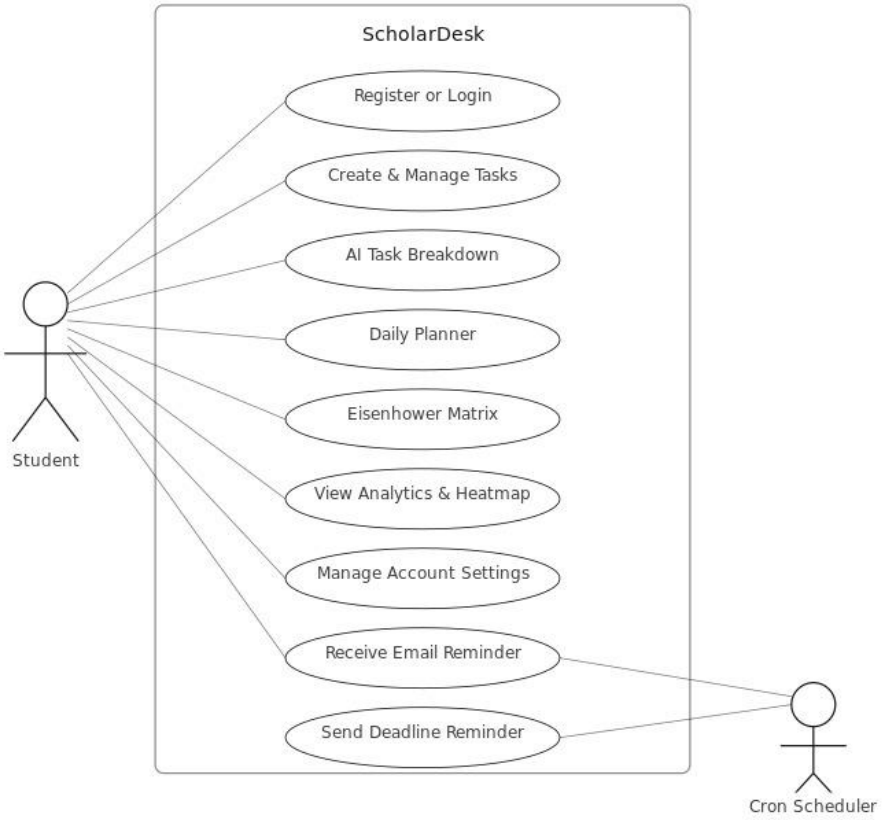


Figure 3: System Use Case Diagram for Tasklytics

The use case diagram in Figure 3 visually represents the dynamic interactions between the system and its two primary actors User and Admin highlighting their respective roles and system access points. It outlines the key functionalities available to each actor: for Users, these include core actions such as Register/Login, Browsing Products, Adding Items to Cart, Making Payments, and Tracking Orders, all of which reflect typical user journeys in an e-commerce environment. For Admins, the diagram showcases essential operational tasks such as Managing Products, Managing Users, and Viewing Reports, emphasizing administrative control over the platform's content and user base. These clearly defined use cases help delineate system boundaries and user permissions, making the diagram an effective tool for identifying functional requirements and user expectations early in the development lifecycle. By presenting this information in a concise, visual format, it serves as a bridge between technical teams and non-technical stakeholders, supporting better collaboration, requirement validation, and informed decision-making throughout the system design process.

5.3 SEQUENCE DIAGRAM

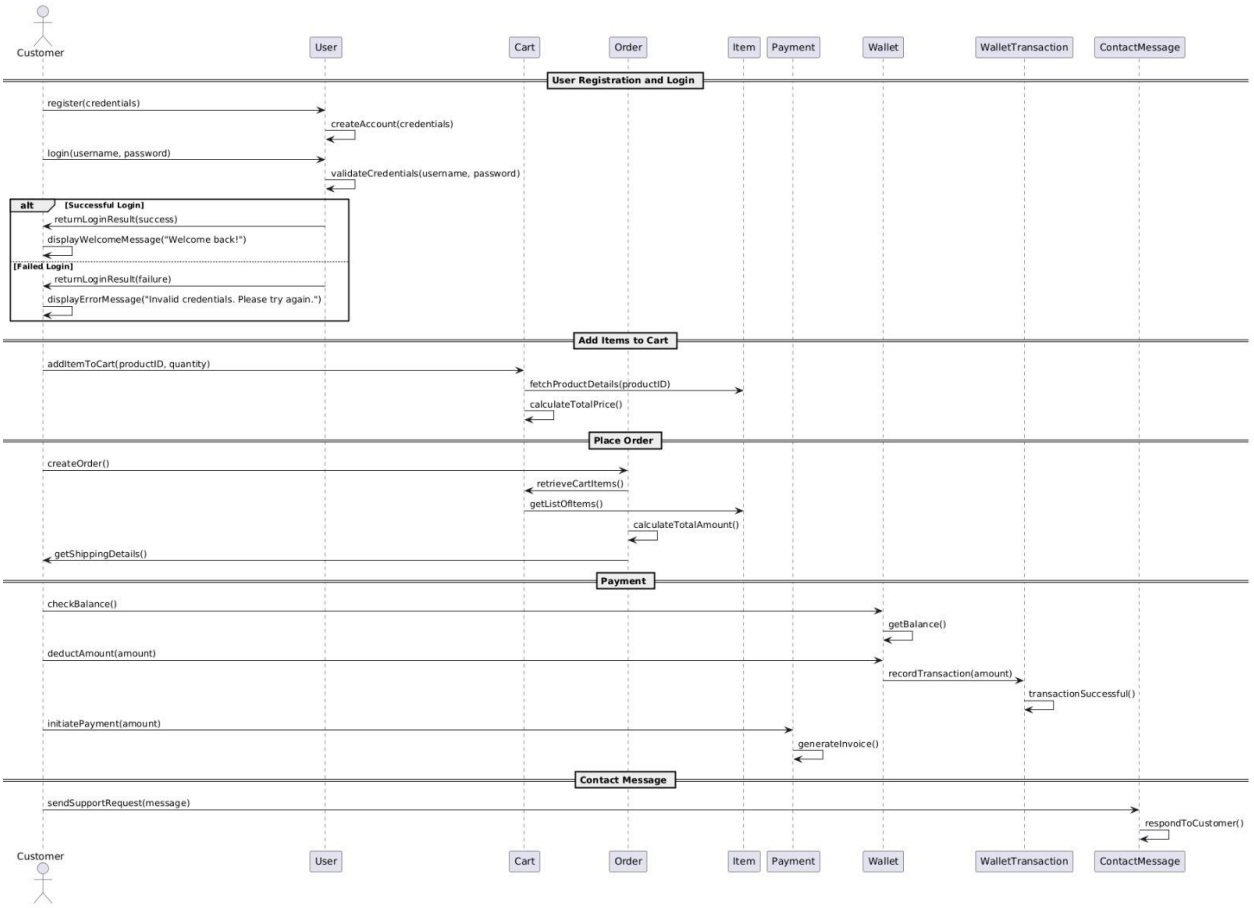


Figure 4: Sequence Diagram of Vivid Skin Co

The sequence diagram in Figure 4 offers a comprehensive and structured view of the dynamic interactions that occur between the system's components over time, specifically focusing on a typical e-commerce order lifecycle. It begins with the Customer—the initiating actor—interacting with the system by registering or logging in, which involves method calls like createAccount() and validateCredentials(). The inclusion of an alt (alternative) control block helps to clearly illustrate branching logic for both successful and failed login attempts, thereby modeling real-world decision-making processes and responses in the application.

Following authentication, the customer proceeds to interact with the Cart and Item components through actions such as addItemToCart() and calculateTotalPrice(), which represent the shopping behavior within the system. The process continues with the creation of an Order, where the system retrieves cart contents using retrieveCartItems() and computes the overall order value through calculateTotalAmount(). This clear chronological layout not only helps in understanding dependencies between components but also assists in mapping user stories directly to implementation tasks.

In the Payment phase, a series of critical backend operations are invoked. The system checks the wallet balance using getBalance(), ensures sufficient funds, and proceeds to deduct the payment amount with deductAmount(). It then records the transaction via the WalletTransaction component and returns a success confirmation. This sequence of actions reinforces financial accuracy, security, and traceability within the platform.

5.4 ACTIVITY DIAGRAM

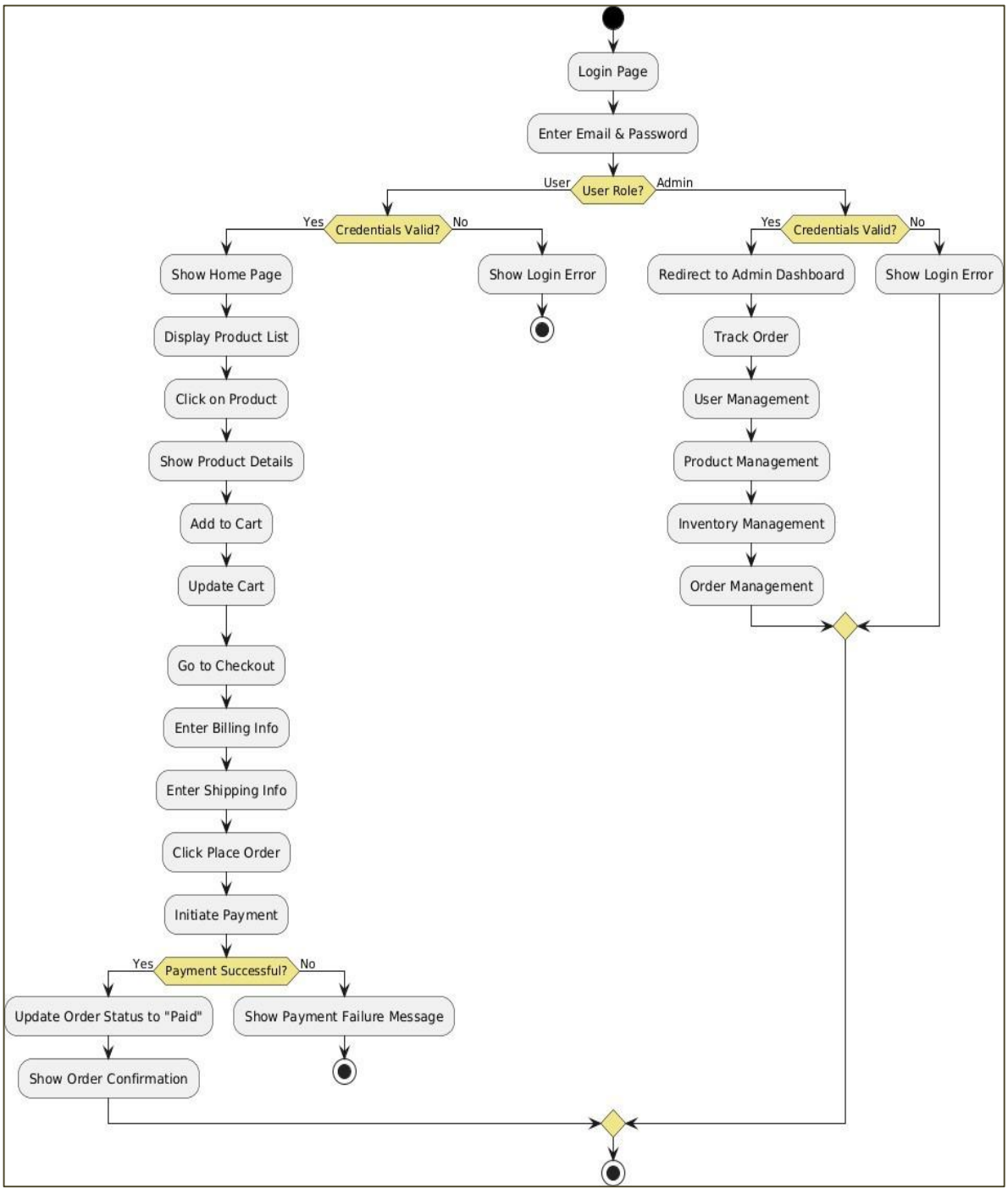


Figure 5: Activity Diagram Illustrating System Workflow

The above Activity Diagram Figure 5, illustrates the workflow of specific processes, such as user checkout or product management. For example, in the checkout process, the diagram shows the sequence of actions: User logs in → Adds products to cart → Reviews cart → Proceeds to payment → Enters shipping details → Confirms payment → Receives order confirmation. It uses actions, decision nodes, and control flows to represent the logic and possible paths. This helps in understanding and optimizing user journeys and backend processes for better usability and system performance.

6. IMPLEMENTATION

6.1 TECHNOLOGY USED

6.1.1 Frontend Modules

1. User Registration & Login Module

This module allows users to create accounts and securely log in to access personalized project management features.

- React.js is used to build interactive login and registration forms with client-side validation and error handling.
- Axios is used to communicate with backend APIs for authentication requests.
- Upon successful login, a JWT token is received from the server and stored securely in local storage.
- The token is used to maintain authenticated sessions and provide access to protected routes and user-specific features.

2. Project & Task Management Module

This module enables users to create, organize, and manage projects and tasks efficiently.

- Users can create projects and divide them into multiple tasks and subtasks.
- React components provide forms and dashboards for task creation, editing, assignment, and deletion.
- Tasks can be categorized using status labels such as Open, In Progress, Blocked, Completed, and Overdue.
- Real-time updates ensure users can easily track task progress and completion.

3. AI Task Breakdown & Time Estimation Module

This module assists users in planning their work more effectively through AI-powered recommendations.

- Users can provide a task description, and the system generates task breakdowns into smaller manageable subtasks.
- Estimated completion times are displayed for each generated subtask.
- Dynamic pages are created for tasks, showing estimated and actual time spent.
- The feature improves project planning and workload management.

4. Productivity Tools Module

This module provides built-in productivity-enhancing tools to help users maintain focus and prioritize work.

- A Pomodoro timer and stopwatch help users track actual working time on tasks.
- The Eisenhower Matrix helps users prioritize tasks based on urgency and importance.
- Users can compare estimated time versus actual time spent on tasks.
- Productivity metrics are displayed through intuitive dashboards and reports.

5. GitHub Activity & Progress Tracking Module

This module visualizes development progress and coding activity.

- GitHub contribution data is displayed using a contribution heatmap similar to GitHub's green activity graph.
- Users can monitor coding consistency and project engagement.
- Visual indicators help track productivity trends over time.
- Progress statistics are presented through charts and summaries.

6. User Dashboard Module

This module serves as the central workspace for users.

- Users can view all projects, tasks, deadlines, and productivity statistics in one place.
- Task completion rates, overdue tasks, and upcoming deadlines are displayed through dashboards.
- Personalized insights help users monitor project progress and performance.
- Responsive design ensures accessibility across different devices.

6.1.2 Backend Modules

1. Authentication & Authorization Module

This module manages secure user authentication and access control.

- Passwords are encrypted using Bcrypt.js before being stored in the database.
- JWT tokens are generated upon successful login and used for session management.
- Middleware verifies token authenticity before allowing access to protected resources.
- Role-based authorization restricts administrative functions to authorized users only.

2. User Management Module

This module handles user-related operations and profile management.

- User details are stored securely in the database.
- APIs allow users to update profile information and account settings.
- User-specific project and productivity data are maintained efficiently.
- The module ensures proper separation of data between users.

3. Project Management Module

This module manages project creation and organization.

- Projects are stored in the database with relevant metadata such as title, description, creation date, and deadlines.
- RESTful APIs enable project creation, modification, retrieval, and deletion.
- Relationships between projects and tasks are maintained efficiently.
- The module provides structured project tracking and management.

4. Task & Subtask Management Module

This module handles all task-related operations.

- Tasks and subtasks are stored with attributes such as priority, status, deadlines, estimated time, and actual time.

- APIs support CRUD operations for tasks and subtasks.
- Automatic status updates help identify overdue tasks.
- Task relationships and dependencies are managed effectively.

5. AI Processing Module

This module powers intelligent task planning features.

- AI services analyze task descriptions and generate meaningful subtasks.
- Time estimation algorithms provide approximate completion durations.
- Generated task structures are stored for future tracking and analysis.
- This module enhances planning accuracy and productivity.

6. Productivity Analytics Module

This module collects and analyzes productivity-related information.

- Pomodoro sessions and stopwatch records are stored in the database.
- Estimated and actual completion times are compared to generate productivity insights.
- Performance statistics and productivity reports are generated dynamically.
- Historical data helps users evaluate work patterns and improve efficiency.

7. GitHub Integration Module

This module manages GitHub activity tracking.

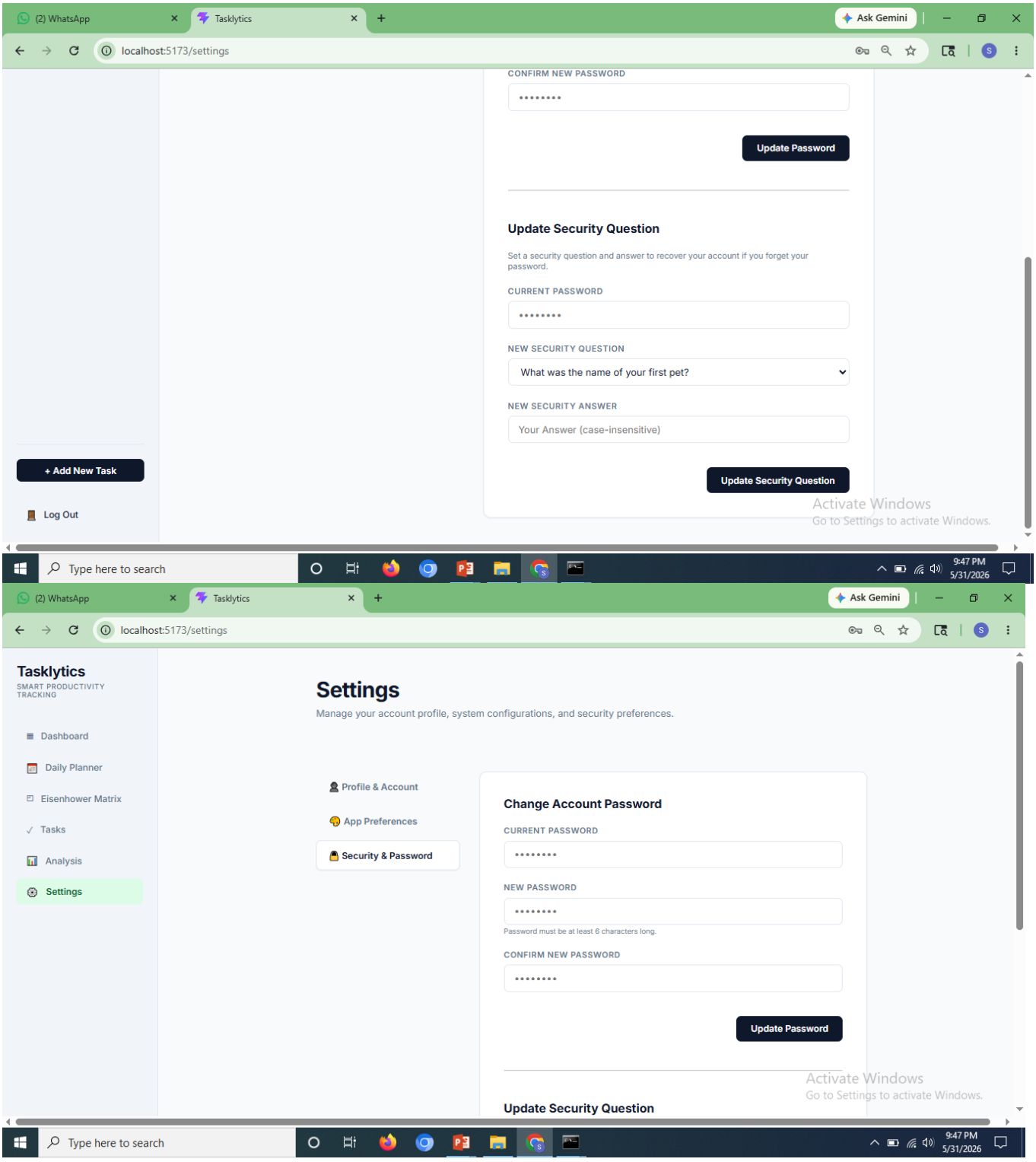
- GitHub APIs are used to retrieve contribution and activity data.
- Contribution information is processed and displayed as heatmaps and progress reports.
- Activity metrics are synchronized periodically to maintain accuracy.
- This module provides visibility into development progress and coding consistency.

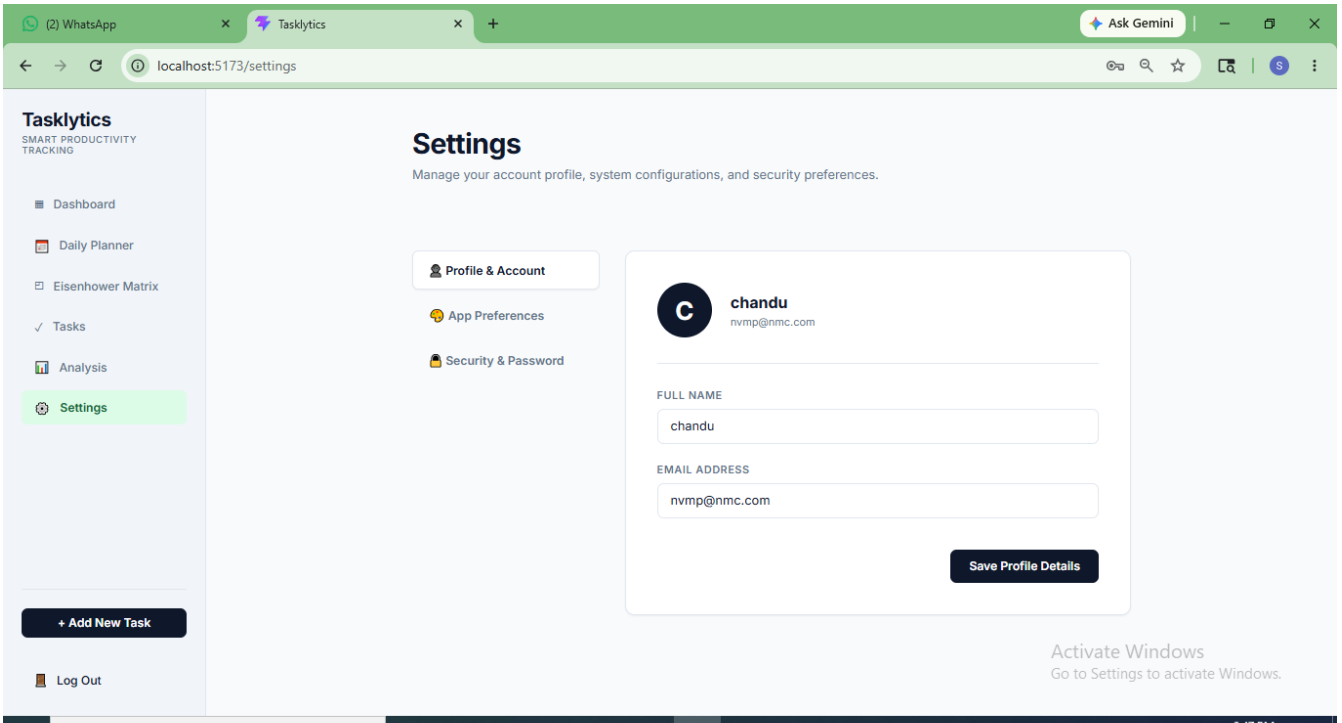
8. Admin Management Module

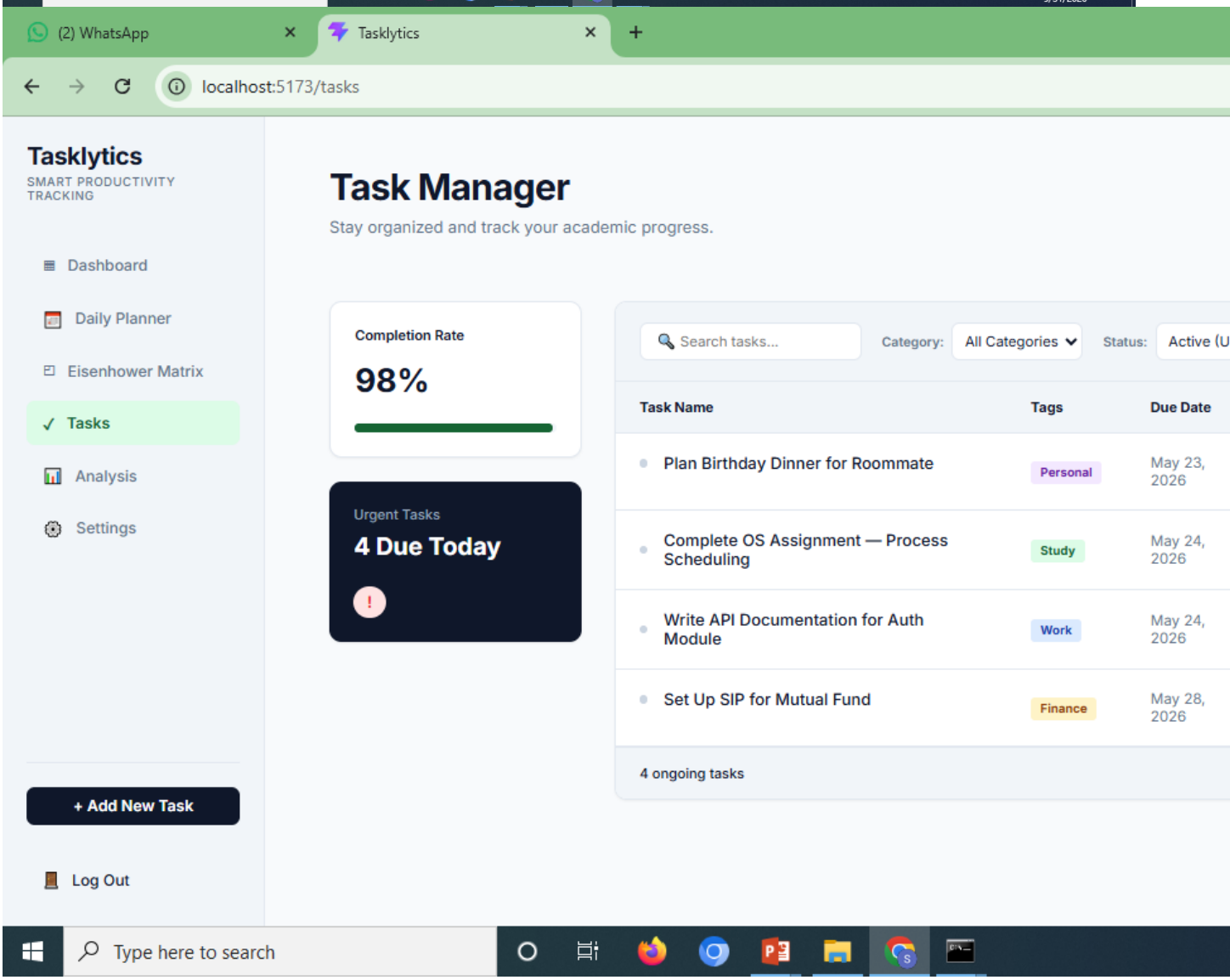
This module provides backend support for administrative operations.

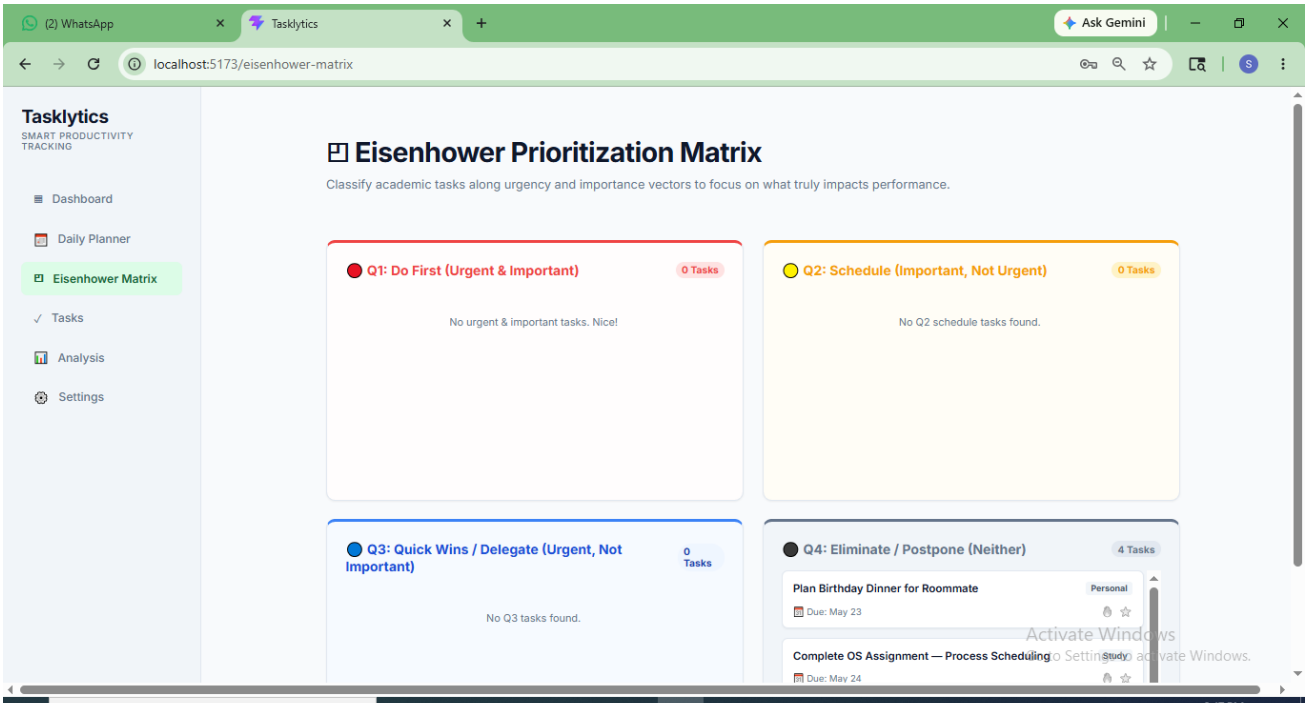
- Administrators can manage users, projects, and platform settings through secured APIs.
- System-wide statistics and reports can be generated for monitoring purposes.
- Access control mechanisms prevent unauthorized administrative actions.
- The module ensures efficient maintenance and operation of the platform.

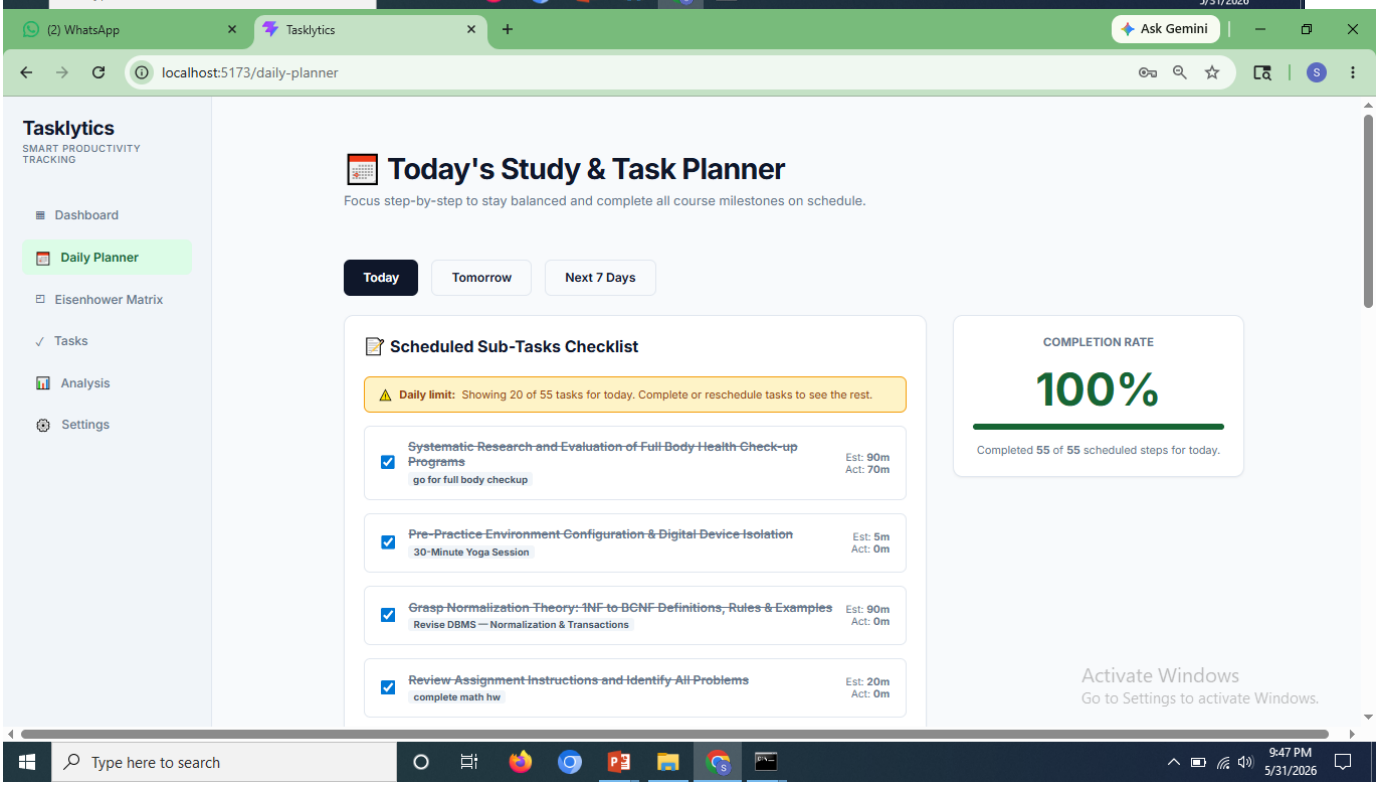
6.2 SCREENSHOTS

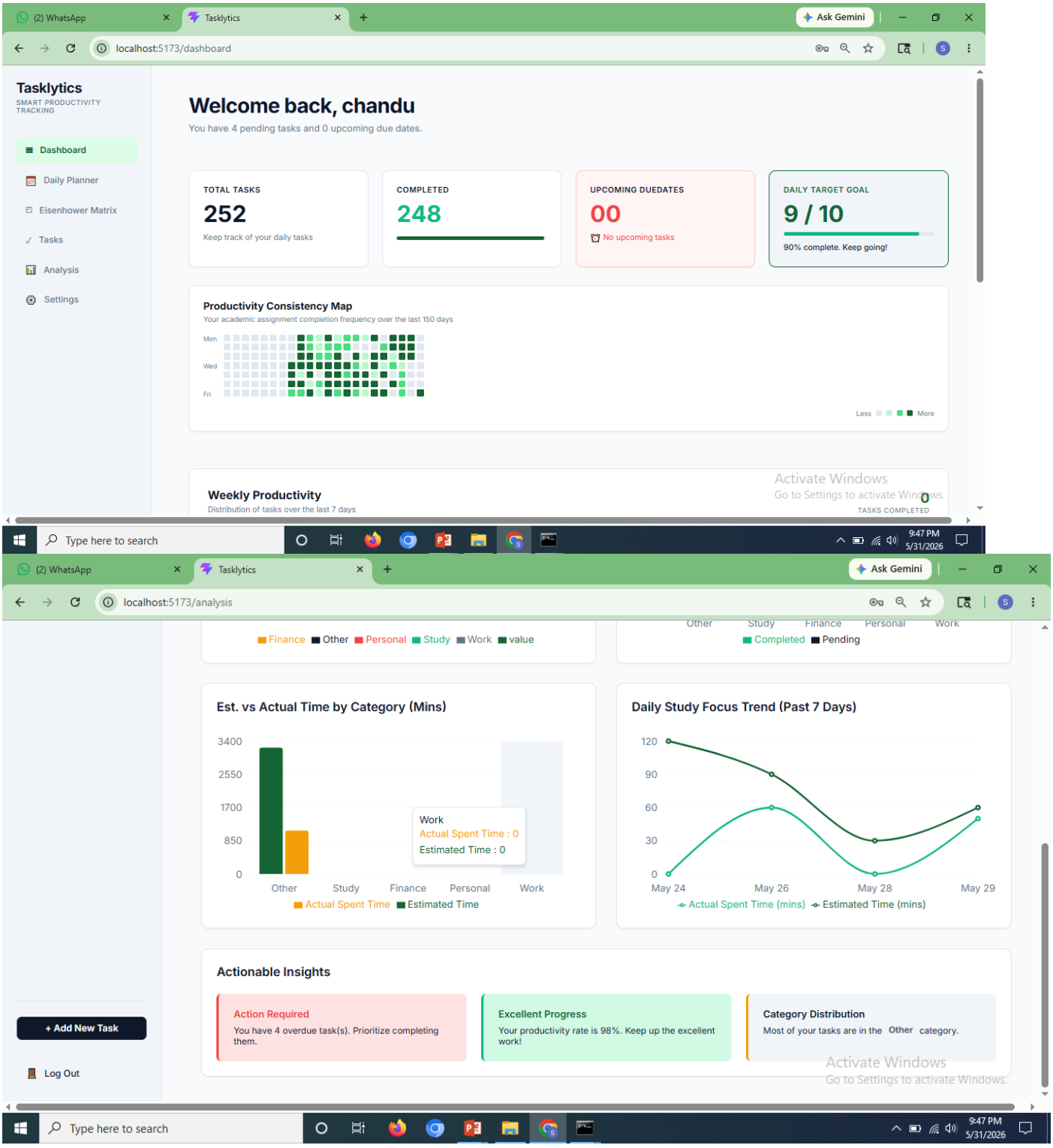



(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/settings
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
+ Add New Task
Log Out
Settings
Manage your account profile, system configurations, and security preferences.
Profile & Account
App Preferences
Security & Password
C
chandu
nvmp@nmc.com
FULL NAME
chandu
EMAIL ADDRESS
nvmp@nmc.com
Save Profile Details
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026

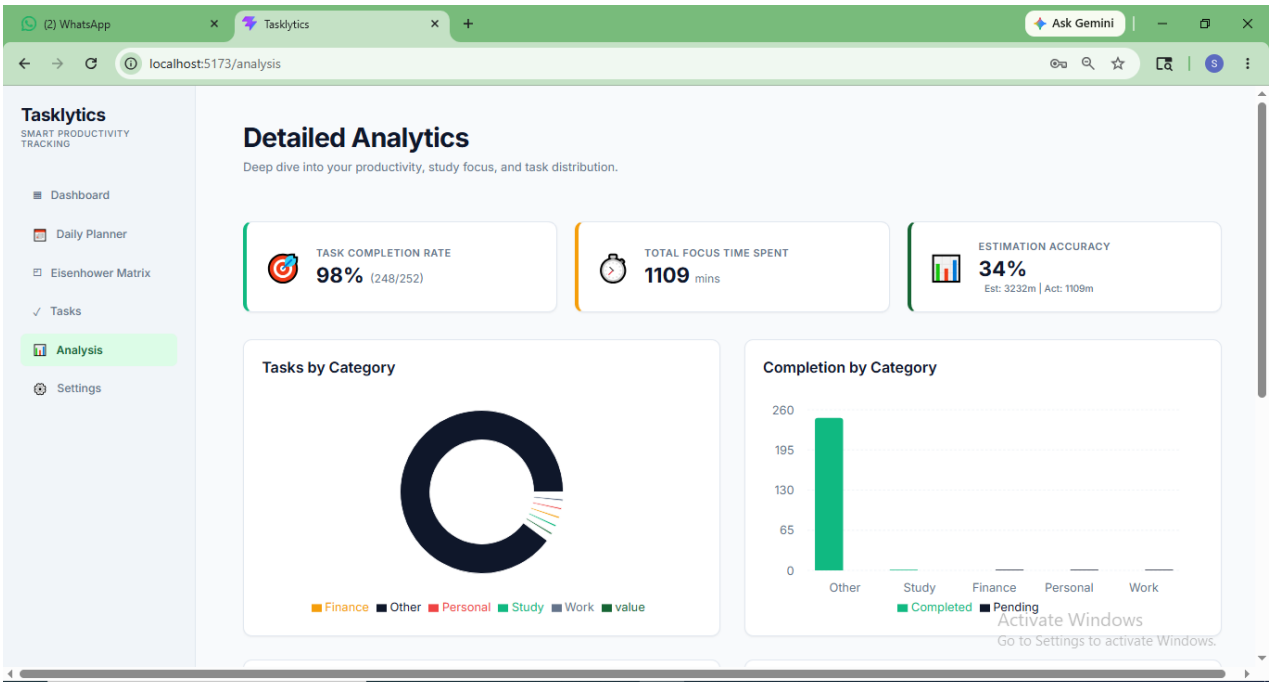

(2) WhatsApp
Tasklytics
localhost:5173/tasks
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
+ Add New Task
Log Out
Task Manager
Stay organized and track your academic progress.
Completion Rate
98%
Urgent Tasks
4 Due Today
Search tasks...
Category:
All Categories
Status:
Active (U
Task Name
Tags
Due Date
Plan Birthday Dinner for Roommate
Personal
May 23, 2026
Complete OS Assignment — Process Scheduling
Study
May 24, 2026
Write API Documentation for Auth Module
Work
May 24, 2026
Set Up SIP for Mutual Fund
Finance
May 28, 2026
4 ongoing tasks
Type here to search

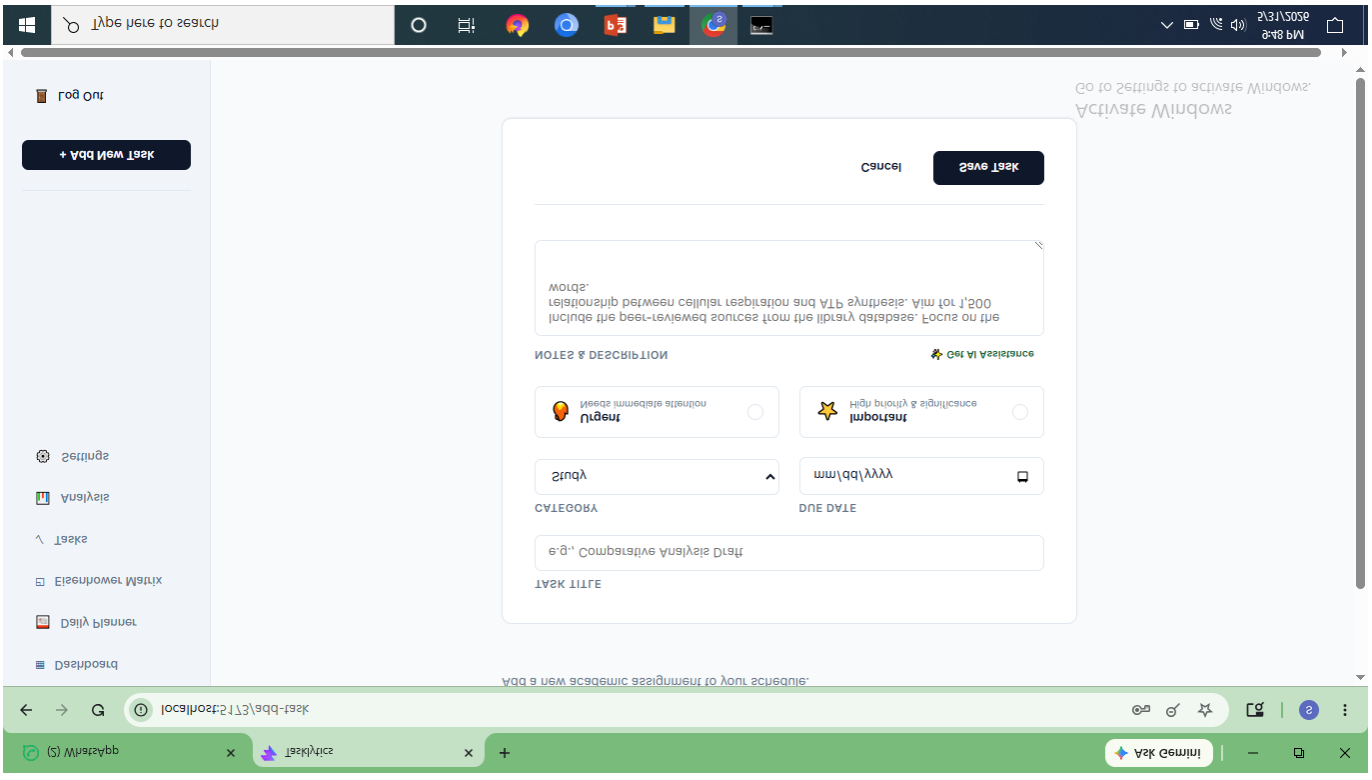

(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/eisenhower-matrix
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
Eisenhower Prioritization Matrix
Classify academic tasks along urgency and importance vectors to focus on what truly impacts performance.
Q1: Do First (Urgent & Important)
0 Tasks
No urgent & important tasks. Nice!
Q2: Schedule (Important, Not Urgent)
0 Tasks
No Q2 schedule tasks found.
Q3: Quick Wins / Delegate (Urgent, Not Important)
0 Tasks
No Q3 tasks found.
Q4: Eliminate / Postpone (Neither)
4 Tasks
Plan Birthday Dinner for Roommate
Personal
Due: May 23
Complete OS Assignment — Process Scheduling
Study
Due: May 24
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026


(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/daily-planner
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
Today's Study & Task Planner
Focus step-by-step to stay balanced and complete all course milestones on schedule.
Today
Tomorrow
Next 7 Days
Scheduled Sub-Tasks Checklist
Daily limit: Showing 20 of 55 tasks for today. Complete or reschedule tasks to see the rest.
Systematic Research and Evaluation of Full Body Health Check-up Programs
go for full body checkup
Est: 90m
Act: 70m
Pre-Practice Environment Configuration & Digital Device Isolation
30-Minute Yoga Session
Est: 5m
Act: 0m
Grasp Normalization Theory: 1NF to BCNF Definitions, Rules & Examples
Revise DBMS — Normalization & Transactions
Est: 90m
Act: 0m
Review Assignment Instructions and Identify All Problems
complete math hw
Est: 20m
Act: 0m
COMPLETION RATE
100%
Completed 55 of 55 scheduled steps for today.
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026


(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/dashboard
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
Welcome back, chandu
You have 4 pending tasks and 0 upcoming due dates.
TOTAL TASKS
252
Keep track of your daily tasks
COMPLETED
248
UPCOMING DUEDATES
00
No upcoming tasks
DAILY TARGET GOAL
9 / 10
90% complete. Keep going!
Productivity Consistency Map
Your academic assignment completion frequency over the last 150 days
Mon
Wed
Fri
Less
More
Weekly Productivity
Distribution of tasks over the last 7 days
TASKS COMPLETED
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026

(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/analysis
Finance
Other
Personal
Study
Work
value
Completed
Pending
Est. vs Actual Time by Category (Mins)
3400
2550
1700
850
0
Work
Actual Spent Time : 0
Estimated Time : 0
Other
Study
Finance
Personal
Work
Actual Spent Time
Estimated Time
Daily Study Focus Trend (Past 7 Days)
120
90
60
30
0
May 24
May 26
May 28
May 29
Actual Spent Time (mins)
Estimated Time (mins)
Actionable Insights
Action Required
You have 4 overdue task(s). Prioritize completing them.
Excellent Progress
Your productivity rate is 98%. Keep up the excellent work!
Category Distribution
Most of your tasks are in the Other category.
+ Add New Task
Log Out
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026


(2) WhatsApp
Tasklytics
Ask Gemini
localhost:5173/analysis
Tasklytics
SMART PRODUCTIVITY TRACKING
Dashboard
Daily Planner
Eisenhower Matrix
Tasks
Analysis
Settings
Detailed Analytics
Deep dive into your productivity, study focus, and task distribution.
TASK COMPLETION RATE
98% (248/252)
TOTAL FOCUS TIME SPENT
1109 mins
ESTIMATION ACCURACY
34%
Est: 3232m | Act: 1109m
Tasks by Category
Finance Other Personal Study Work value
Completion by Category
260
195
130
65
0
Other Study Finance Personal Work
Completed Pending
Activate Windows
Go to Settings to activate Windows.
Type here to search
9:47 PM
5/31/2026



7. TESTING

7.1 User Registration & Login Module

This module tests the core user authentication functionalities shown in Table 4, ensuring that users can register with valid details, log in securely, and access protected routes appropriately. It also validates error handling for invalid inputs, duplicate registrations, and unauthorised access attempts.

Table 4 User Registration & Login Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-001	Register with valid inputs	Valid name, email, password	Account should be created	Account created successfully	PASS
TC-002	Register with existing email	Duplicate email	Show "Email already exists" error	Error displayed correctly	PASS
TC-003	Login with valid credentials	Correct email & password	User logged in and redirected to dashboard	Redirected to dashboard	PASS
TC-004	Login with wrong password	Valid email, wrong password	Show "Invalid credentials" message	Error message shown	PASS
TC-005	Access protected page without login	No token	Redirect to login page	Redirected as expected	PASS
TC-006	Register with invalid email format	Name, invalid email (e.g., user@com)	Show "Invalid email format" error	Error message shown	PASS
TC-007	Login with unregistered email	Unregistered email, any password	Show "User not found" error	Error message shown	PASS
TC-008	Registration with missing required fields	Missing password	Show "Password is required" error	No error, registration allowed	FAIL
TC-008a	(Fix) Validation added for missing fields	Missing password	Show "Password is required" error	Error message shown	PASS
TC-009	Login with empty email and password	Blank email and password	Show "Please fill in all fields" error	Error shown correctly	PASS

7.2 Task Management Module

This module focuses on ensuring that users can effectively create, view, update, filter, and delete academic tasks. The tests cover key functionalities including task creation with valid and invalid inputs, filtering by category and priority, editing task details, and handling edge cases such as missing required fields or unauthorised access. The goal is to provide a reliable and intuitive task management experience. Table 5 below summarises the test cases executed for this module.

Table 5 Test Cases for Task Management Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-011	Add task with valid inputs	Valid title, category, due date,	Task created and shown in list	Task added successfully	PASS

		priority			
TC-012	Add task with missing title	Empty title field	Show "Title is required" error	Error message shown	PASS
TC-013	Filter tasks by category	Select course category	Only tasks of that category displayed	Filtered correctly	PASS
TC-014	Filter tasks by priority	Select "High" priority	Only high-priority tasks shown	Tasks filtered correctly	PASS
TC-015	Edit an existing task	Update title and due date	Task updated and reflected in list	Task updated successfully	PASS
TC-016	Delete a task	Click delete on a task	Task removed from list	Task deleted successfully	PASS
TC-017	View task with no matching filter	Apply filter with no matches	Show "No tasks found" message	No tasks message displayed	PASS
TC-018	Access another user's task	Different user ID in request	Return 404 or unauthorised error	Unauthorised error returned	PASS

7.3 Subtask & AI Breakdown Module

Table 6 Test Cases for Subtask & AI Breakdown Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-019	Add subtask to a task	Valid subtask title and time	Subtask saved and displayed	Subtask added successfully	PASS
TC-020	Mark subtask as complete	Click complete checkbox	Subtask marked complete	Status updated correctly	PASS
TC-021	Delete a subtask	Click delete on subtask	Subtask removed from list	Subtask deleted successfully	PASS
TC-022	Generate AI breakdown with valid task	Task with title, description, category	4–6 subtasks, tip, and effort level returned	AI breakdown displayed correctly	PASS
TC-023	Generate AI breakdown with missing description	Task with title only	Breakdown generated using available fields	Breakdown returned with defaults	PASS
TC-024	AI breakdown with API unavailable	Gemini API key absent	Fallback subtasks returned	Default subtasks shown	PASS

7.4 Daily Planner Module

Table 7 Test Cases for Daily Planner Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-025	View daily subtasks for current date	Logged-in user	Subtasks due today displayed	Correct subtasks shown	PASS
TC-026	Mark daily subtask complete from planner	Click complete	Subtask status updated	Status updated correctly	PASS
TC-027	View planner with no tasks due today	No subtasks scheduled	Show "No tasks for today" message	Message displayed correctly	PASS

7.5 Analytics & Heatmap Module

Table 8 Test Cases for Analytics & Heatmap Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-028	Load analytics dashboard	Logged-in user with tasks	Charts render with correct data	Charts displayed correctly	PASS
TC-029	View heatmap with activity	User with completed tasks	Activity grid shows correct density	Heatmap rendered correctly	PASS
TC-030	View analytics with no tasks	New user with no data	Show empty state or zero values	Empty state shown correctly	PASS

7.6 Email Reminder Module

Table 9 Test Cases for Email Reminder Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-031	Cron job triggers for tasks due tomorrow	Task with due date = tomorrow	Reminder email dispatched to user	Email received correctly	PASS
TC-032	No email sent when no tasks due tomorrow	No matching tasks in DB	No email dispatched	No email sent as expected	PASS
TC-033	Email content correctness	Task title and category in DB	Email contains correct task name and due date	Correct content in email	PASS

7.7 User Profile & Settings Module

Table 10 Test Cases for User Profile & Settings Module

Test ID	Test Case Description	Input	Expected Result	Actual Output	Status
TC-034	View user profile	Logged-in user	Show name, email, and profile details	Profile displayed correctly	PASS
TC-035	Update profile information	Edit form and submit	Info updated and reflected	Profile updated successfully	PASS
TC-036	Change password with correct old password	Old + new password	Password updated successfully	Password changed	PASS
TC-037	Change password with incorrect old password	Wrong old password	Show "Current password incorrect" error	Error message shown	PASS
TC-038	Update security question	New question and answer	Security question updated	Updated successfully	PASS

8. Impact on Society and Environment

Impact on Society

Tasklytic significantly contributes to enhancing the way students plan, manage, and execute their academic responsibilities. By offering a digital platform that is both intelligent and user-friendly, it empowers individuals to make better-informed decisions regarding their study habits and time allocation. The inclusion of features such as AI-generated task breakdowns, personalised success strategies, a daily planner, and automated deadline reminders makes academic productivity more accessible — especially for students who struggle with self-organisation or do not have access to academic coaching or tutoring support.

The platform supports the growing self-improvement and study-wellness movement by promoting structured planning, consistent effort tracking, and awareness of workload distribution. The Eisenhower Matrix and productivity heatmap encourage students to reflect on their priorities and study patterns, fostering habits that extend well beyond the platform itself. By centralising task information and providing intelligent guidance, Tasklytic also helps reduce academic stress and the anxiety associated with missed deadlines or poorly managed workloads.

From a broader perspective, the project demonstrates how AI technology can be democratised and applied meaningfully in educational contexts without requiring expensive institutional infrastructure. Students from any background can access the same quality of AI-assisted academic planning that would otherwise require paid productivity tools or personal tutoring. From a developer's standpoint, this project also contributes to real-world learning, building valuable skills in full-stack development, AI API integration, database design, and software engineering best practices.

Impact on Environment

Although digital platforms generally have a lower direct environmental impact compared to physical infrastructure, the environmental implications of running cloud-based web applications must still be considered. By shifting academic planning and task management entirely online, Tasklytic eliminates the need for printed planners, paper-based to-do systems, and physical study organisers — contributing to reduced paper consumption and waste associated with traditional academic planning tools.

The platform's automated email reminder system replaces the need for printed reminder sheets or physical notice boards, and its cloud-based deployment on platforms such as Vercel minimises the energy overhead associated with maintaining dedicated on-premises servers. The serverless and efficient architecture of the application ensures that computational resources are consumed only when needed, reducing idle energy usage.

However, the environmental cost of AI API calls — particularly to large language models such as Google Gemini — must be acknowledged, as inference on large models carries an energy footprint. To address this responsibly, the project incorporates a deterministic fallback mechanism that avoids unnecessary API calls when the service is unavailable, and prompts are structured to be concise and targeted, minimising token usage per request.

Future enhancements to Tasklytic could further reduce its environmental footprint by:

- Implementing response caching for commonly requested AI breakdowns to avoid redundant API calls,
- Optimising database query efficiency to reduce server processing load,
- Adopting green hosting providers that operate on renewable energy infrastructure,
- Encouraging digital-first study habits that reduce dependence on printed materials across academic institutions.

By promoting structured digital academic management and making responsible engineering decisions, Tasklytic serves as a step toward balancing technological convenience, intelligent personalisation, and environmental responsibility in the educational software space.

.

9. CONCLUSION

The development of Tasklytic — an AI-powered academic task management system — aimed to address the limitations found in generic productivity tools by offering a tailored, intelligent, and performance-optimised solution designed specifically for students. The platform provides structured task and subtask management, AI-generated study plans, visual productivity analytics, and automated deadline notifications, catering to the unique needs of academic users who require more than a simple to-do list to manage their coursework effectively.

Technically, the system incorporates user authentication, full CRUD task and subtask management, Google Gemini AI integration, a daily planner, Eisenhower matrix prioritisation, a productivity heatmap, and a cron-based email reminder service. Using React.js with a component-based architecture enabled code reusability and simplified maintenance, while a fully responsive design ensured a consistent experience across all devices. On the backend, Node.js and Express.js provided a flexible, non-blocking server environment, with MySQL handling structured relational data efficiently. JWT-based authentication and secure API design safeguarded user information, and modular routing across user, task, analytics, and AI domains improved scalability and long-term maintainability.

The project introduced valuable real-world experience in full-stack development, RESTful API design, AI API integration, database modelling, scheduled background services, and cloud deployment. Comprehensive testing was conducted throughout the development cycle, including unit tests for individual components, API testing via Postman, and integration testing to validate interactions between the frontend and backend systems. These practices ensured system reliability and a smooth user experience across devices and browsers.

Overall, Tasklytic strengthened technical skills across the team and demonstrated how thoughtful design, intelligent feature integration, and robust engineering can result in a professional-grade, scalable academic productivity platform. The project not only fulfilled its immediate functional goals but also laid a solid foundation for ongoing evolution in the academic technology space.

10. FUTURE ENHANCEMENTS

1. Conversational AI Chatbot

Integrating a conversational study assistant using tools such as the Gemini API, ChatGPT API, or Dialogflow would allow students to interact with the platform in natural language — asking questions about their tasks, requesting study tips, or getting instant breakdowns without navigating through the UI manually. An in-app chatbot could also provide motivational nudges, answer academic queries, and guide new users through the platform's features, improving engagement and onboarding significantly.

2. Mobile Application Development

Developing a dedicated mobile application using Flutter or React Native would enhance accessibility and convenience for students who primarily use smartphones for study management. Features such as push notifications for deadline reminders, offline task viewing, and biometric authentication would improve usability and encourage consistent daily engagement with the platform.

3. Collaborative Study Groups

A future version of Tasklytic could support group task management, allowing students to collaborate on shared assignments, divide subtasks among group members, and track collective progress in real time. Role-based access within groups — such as group owner, editor, and viewer — would provide structured collaboration aligned with academic team projects.

4. Integration with Learning Management Systems (LMS)

Connecting Tasklytic with institutional platforms such as Moodle, Google Classroom, or Canvas would allow tasks and deadlines to be automatically imported from a student's enrolled courses. This would eliminate manual task entry, reduce duplication across tools, and ensure the platform reflects the student's actual academic schedule in real time.

5. Advanced Analytics and Personalised Insights

Implementing deeper analytics capabilities — such as subject-level performance trends, predicted deadline risk scoring, and weekly study consistency reports — would provide students with actionable, data-driven insights into their academic habits. Integration with tools such as Google Analytics or custom reporting dashboards would also allow administrators or institutions to monitor platform usage and identify areas for improvement.

6. Subscription and Institutional Licensing

A tiered subscription model could unlock premium features such as unlimited AI breakdowns, advanced analytics, group collaboration, and LMS integration. An institutional licensing option would allow universities or schools to deploy Tasklytic as a white-labelled productivity platform for their students, expanding the platform's reach and sustainability as a long-term product.

12. BIBLIOGRAPHY

- Express.js Foundation. (n.d.). *Express Documentation.* Retrieved from https://expressjs.com
- React Official Documentation – Meta. (n.d.). *React Docs – A JavaScript library for building user interfaces.* Retrieved from https://react.dev
- Node.js Documentation – OpenJS Foundation. (n.d.). *Node.js Documentation.* Retrieved from https://nodejs.org/en/docs
- MySQL Documentation – Oracle Corporation. (n.d.). *MySQL 8.0 Reference Manual.* Retrieved from https://dev.mysql.com/doc
- mysql2 – npm Registry. (n.d.). *mysql2 – Fast MySQL driver for Node.js.* Retrieved from https://www.npmjs.com/package/mysql2
- Google Generative AI – Google LLC. (n.d.). *Gemini API Documentation.* Retrieved from https://ai.google.dev/docs
- Nodemailer – Nodemailer. (n.d.). *Nodemailer – Send emails from Node.js.* Retrieved from https://nodemailer.com
- node-cron – npm Registry. (n.d.). *node-cron – Task scheduler for Node.js.* Retrieved from https://www.npmjs.com/package/node-cron
- JSON Web Tokens – Auth0. (n.d.). *Introduction to JSON Web Tokens.* Retrieved from https://jwt.io/introduction
- bcrypt – npm Registry. (n.d.). *bcrypt – Password hashing library for Node.js.* Retrieved from https://www.npmjs.com/package/bcrypt
- Vite – Evan You & Vite Contributors. (n.d.). *Vite – Next Generation Frontend Tooling.* Retrieved from https://vitejs.dev
- W3Schools – W3Schools.com. (n.d.). *HTML, CSS, JavaScript Tutorials.* Retrieved from https://www.w3schools.com
- MDN Web Docs – Mozilla Developer Network. (n.d.). *Web technology reference.* Retrieved from https://developer.mozilla.org
- GitHub – GitHub Inc. (n.d.). *Open-source project collaboration and version control.* Retrieved from https://github.com
- GeeksforGeeks. (n.d.). *Full Stack Development Tutorials and Concepts.* Retrieved from https://www.geeksforgeeks.org
- Figma – Figma Inc. (n.d.). *Collaborative Interface Design Tool. Used for UI wireframing and prototypes.* Retrieved from https://www.figma.com